



OpenCode

Manual

Tools · Skills · Plugins · Commands · MCP · Agentes

¿Qué es OpenCode?

Un agente de IA para programar, en la terminal y en tu editor.



Agente, no chatbot

No solo responde: lee tu proyecto, edita archivos, ejecuta comandos e itera hasta terminar.



Model-agnóstico

Funciona con 75+ proveedores (Groq, OpenRouter, OpenAI, local...). Cambias de modelo en segundos.



Abierto y tuyo

Open source, terminal + VS Code + app. Tus conversaciones se guardan en local.

Cómo trabaja por dentro

El bucle del agente, aplicado a tu código.



Las acciones que ejecuta (leer, editar, ejecutar...) son sus HERRAMIENTAS. Es el function calling del curso, ya hecho.

Puesta en marcha (resumen)

Tres pasos para tenerlo listo.



1. Instalar

Windows → WSL 2
recomendado. Script: `curl -fsSL
https://opencode.ai/install |
bash`. No necesita entorno
virtual.



2. Conectar modelo

Dentro: `/connect` → Groq (u
OpenRouter) → pega tu clave.
Elige modelo con `/models`.



3. Inicializar

`/init` analiza el proyecto y crea
AGENTS.md (sus normas).

Plan vs. Build, y los permisos

El flujo seguro de trabajo profesional. Cambias con la tecla Tab.



Modo Plan (solo lectura)

Analiza y propone CÓMO lo haría, sin tocar nada. Ideal para diseñar antes de ejecutar.



Modo Build (por defecto)

Hace los cambios de verdad: crea/edita archivos y ejecuta comandos.



Permisos = tu red de seguridad. Cada herramienta puede ser allow / ask / deny. En equipo, empieza con bash y edit en "ask".

Cómo se potencia OpenCode

Seis piezas que conviene NO confundir. Las vemos una a una.



Tools

Acciones que ejecuta



Skills

Instrucciones reutilizables



Plugins

Código que lo extiende



Commands

Atajos de prompts



MCP

Herramientas externas



Agentes/Rules

Configurar el asistente

Tools — las herramientas

Las ACCIONES que el agente ejecuta en tu proyecto.

Integradas (siempre disponibles):

read · write · edit · apply_patch

bash (ejecutar comandos)

grep · glob (buscar texto / archivos)

lsp (inteligencia de código)

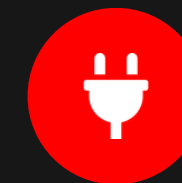
webfetch · websearch

skill · todowrite · question



Se configuran con permisos

En opencode.json: allow / ask / deny por herramienta.



Se amplían

Con plugins (custom tools) y con servidores MCP.

Skills — el "saber hacer"

Instrucciones reutilizables que el agente carga cuando las necesita.



Qué son

Un "manual" de cómo abordar una tarea (estilo de docstrings, cómo hacer un release...). Conocimiento, no código.



Carga bajo demanda

El agente ve la lista y abre la skill solo cuando encaja. Ahorra contexto.

Se configuran como archivos SKILL.md:

Ubicación: `.opencode/skills/<nombre>/SKILL.md` (o global en `~/.config/opencode/skills/`)

Contenido: frontmatter YAML (name, description) + instrucciones en Markdown

Plugins — extender OpenCode

Código (JS/TS) que se "engancha" a los eventos de OpenCode.



Qué hacen

Modifican el comportamiento: enganchan eventos (antes/después de una herramienta, al guardar...).



Para qué sirven

Avisos al terminar, proteger .env, inyectar variables, añadir herramientas propias, integrar servicios.

Cómo se instalan:

Locales: archivos .ts/.js en .opencode/plugins/

De npm: en opencode.json → "plugin": ["opencode-wakatime", "@org/mi-plugin"]

Commands y MCP

Dos piezas más para tu flujo profesional.



Commands (atajos de prompt)

Prompts frecuentes guardados como .md en .opencode/commands/. El archivo tests.md crea el comando /tests. Útil para tareas repetitivas del equipo.



MCP servers

Conectan herramientas externas vía el protocolo MCP : bases de datos, APIs, sistema de archivos... Se declaran en opencode.json.

Agentes y reglas del proyecto

Cómo configuras al asistente y su contexto.



Agentes (Build · Plan · subagentes)

Build (acceso total) y Plan (solo lectura). Puedes crear subagentes especializados con su prompt, modelo y permisos.



Rules — AGENTS.md

Las normas permanentes del proyecto: estilo, convenciones, qué hacer y qué no. Se versiona en Git.



Clave en equipo: guarda AGENTS.md, skills y commands EN EL REPOSITORIO. Así todo el equipo comparte el mismo asistente.

Diferencias de un vistazo

La misma pregunta para cada pieza: ¿qué es, dónde se configura y para qué?

Concepto	Qué es	Dónde se define	Para qué
Tool	Una acción que ejecuta	Integradas + permisos	Hacer (leer, editar, bash)
Skill	Instrucciones (saber cómo)	.opencode/skills/...SKILL.md	Enseñar a abordar tareas
Plugin	Código que engancha eventos	.opencode/plugins/ o npm	Cambiar/ampliar el agente
Command	Un prompt reutilizable	.opencode/commands/*.md	Atajos de instrucciones
MCP	Proveedor de tools externo	opencode.json (mcp)	Conectar apps y datos
Agente/Rules	Asistente y sus normas	Tab · agents/ · AGENTS.md	Modos y contexto del proyecto

¿Cuándo uso cada uno?

La regla rápida para no equivocarte.



Quiero que HAGA algo (leer, editar, ejecutar)

→ **Tools (y sus permisos)**



Quiero enseñarle CÓMO hacer una tarea a mi manera

→ **Skill (SKILL.md)**



Quiero cambiar su comportamiento o integrarlo

→ **Plugin**



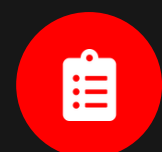
Repito un mismo prompt muy a menudo

→ **Command (/atajo)**



Necesito conectar datos o apps externas

→ **MCP server**



Quiero fijar normas y estilo del proyecto

→ **AGENTS.md (Rules)**

Dónde vive la configuración

Todo se centraliza en el proyecto (y se puede tener global).

<code>opencode.json</code>	→ modelo, permisos, plugins (npm), MCP
<code>AGENTS.md</code>	→ normas y contexto del proyecto
<code>.opencode/</code>	
<code>skills/<nombre>/SKILL.md</code>	→ skills
<code>commands/<nombre>.md</code>	→ comandos (/nombre)
<code>plugins/*.ts</code>	→ plugins locales
<code>agents/<nombre>.md</code>	→ subagentes
<code>~/.config/opencode/</code>	→ versión GLOBAL de todo lo anterior

Regla pro: lo del proyecto se versiona en Git (compartido); lo global es tu configuración personal.

Buenas prácticas en el trabajo

Para usarlo con seguridad y en equipo.



Permisos primero

Empieza con bash y edit en "ask". Revisa los diffs antes de aceptar.



Compártelo en el repo

AGENTS.md, skills y commands en Git: mismo asistente para todo el equipo.



Seguridad

Protege secretos (plugin anti-.env), y conecta solo MCP de confianza.



Controla el coste

Un modelo por tarea: rápido/barato para lo simple, potente para lo complejo.

En resumen



Las ideas que te llevas.



OpenCode = un agente de programación, con el modelo que elijas (Groq en el curso).



Tool = HACER · Skill = SABER CÓMO · Plugin = EXTENDER el agente.



Command = atajo de prompt · MCP = herramientas externas · Rules = normas.



Casi todo se configura en `opencode.json` y la carpeta `.opencode/`.



En equipo: permisos en "ask", y todo versionado en Git.